



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Ruanfeng Lin
August 8, 2026



Presentation Outline

Executive Summary & Core Objectives

Methodology: Data Collection & Feature Engineering

Exploratory Data Analysis (EDA) via SQL Queries

Interactive Geospatial Analysis Using Folium Maps

Dynamic Visualizations via Plotly Dash App

Predictive Machine Learning Modeling & Results

Project Conclusion & GitHub Repository Link

Executive Summary

Methodological Implementation and Pipeline:

Collected and cleaned SpaceX API data, performed EDA with SQL and visualization, applied StandardScaler, and tuned 4 classification models via GridSearchCV.

Predictive Modeling Performance and Results:

Logistic Regression, SVM, and KNN achieved the highest test accuracy of 83.33%, while Decision Tree scored 72.22%.

Introduction

SpaceX Falcon 9 Mission Background:

SpaceX Falcon 9 launch costs are drastically reduced due to the reusability of the first-stage rocket.

Determining whether the first stage will land successfully allows for accurate commercial price estimation.

Core Predictive Objectives and Questions:

Can we accurately predict the successful landing of the Falcon 9 first stage using machine learning models?

Which classification algorithm provides the most reliable predictions based on flight data?

Section 1

Methodology

Methodology

Data Collection Methodology:

Collected flight data via SpaceX API and scraped launch site information from Wikipedia.

Data Wrangling Pipeline:

Cleaned missing values, created target label Class (1 for success, 0 for failure), and applied One-Hot Encoding to categorical features.

Exploratory Data Analysis (EDA):

Conducted rigorous exploratory data analysis using data visualization libraries and advanced SQL queries.

Interactive Visual Analytics:

Developed interactive geospatial visualization models utilizing Folium and dynamic Plotly Dash web applications.

Predictive Analytics and Modeling:

Built and evaluated machine learning models using StandardScaler data pipeline. Optimized hyperparameters for Logistic Regression, SVM, Decision Tree, and KNN via 10-fold GridSearchCV to deliver the highest prediction accuracy.

Data Collection

Data Collection Sources and Methodologies:

- SpaceX API Integration: Requested raw JSON records via canonical API endpoints to fetch precise historic flight numbers, dates, payloads, and launch outcomes.
- Web Scraping via BeautifulSoup: Scraped supplemental launch site geographic details and orbital infrastructure information directly from Wikipedia tables.

Core Pipeline and Transformations:

- Structured Tabular Output: Flattened dense JSON array formats, isolated critical features, and exported a unified Pandas dataframe for downstream exploratory data analysis.

Data Collection – SpaceX API

API Endpoint: <https://spacexdata.com>

Core Process: Requested raw JSON records -> Flattened payload and core dictionary arrays -> Screened essential missing values -> Output structured tabular dataframe.

Key Fields Extracted: FlightNumber, Date, PayloadMass, Orbit, LaunchSite, Core, and landing outcomes.

Code Repository:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

```
import requests
import pandas as pd

# 1. Request SpaceX static backup data from IBM Cloud
static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_api.json'

response = requests.get(static_json_url)

# 2. Check response status and decode nested JSON structure
if response.status_code == 200:
    data = pd.json_normalize(response.json())

# 3. Verify data loading success and check matrix shape
print("Data loading completed successfully.")
print(data.shape)
```

Data Collection - Scraping

Target URL: <https://wikipedia.org>

Core Process: Dispatched an HTTP GET request → Captured raw Wikipedia HTML source content → Utilized BeautifulSoup parser to scan target table tags → Isolated structured text rows.

Key Fields Extracted: FlightNumber, Date, BoosterVersion, LaunchSite, Payload, PayloadMass, Orbit, Customer, and LaunchOutcome.

Code Repository:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

# 1. Fetch historical launch records from full Wikipedia source URL
wiki_url = "https://wikipedia.org"
response = requests.get(wiki_url)

# 2. Parse structural text fields using BeautifulSoup elements
soup = BeautifulSoup(response.text, 'html.parser')
html_tables = soup.find_all('table')

# 3. Web scraping task initialized successfully
print("Successfully parsed HTML tables from Wikipedia source.")
print(len(html_tables))
```

Data Wrangling

Data Processing Methodology:

- Explored training columns and identified the distributions of launch sites ('CCAFS SLC-40', 'KSC LC-39A', 'VAFB SLC-4E') and orbit types.
- Analyzed the 'Outcome' column using `value_counts()` to determine landing success trends.
- Created the target label 'Class' by mapping successful recovery types (True RTLS, True ASDS, True Ocean) to 1, and unsuccessful/none outcomes (False RTLS, False ASDS, None) to 0.

Data Wrangling Process Flowchart:

[Raw API/Scraped Data] —> [Identify Missing Values] —> [Analyze Outcomes via `value_counts()`] —> [Derive Binary 'Class' Column (0 or 1)] —> [Export Structured Tabular CSV Dataset] Completed

Data Wrangling Related Notebook URL:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

EDA with Data Visualization

Summarized Chart Descriptions and Plotting Objectives:

- Scatter Plots (FlightNumber vs. PayloadMass): Utilized to map flight sequences against payload sizes to uncover historical success rate evolution across launch sites.
- Scatter Plots (FlightNumber vs. LaunchSite): Plotted to examine operational distributions and discover specific historical launch cluster frequencies over time.
- Bar Charts (Success Rate vs. Orbit Type): Plotted to aggregate categorical values and identify the highest performing payload orbit types (such as ES-L1, GEO, HEO, SSO).
- Line Charts (Success Rate Trends vs. Year): Generated to trace temporal trends and visualize year-over-year performance improvements due to technological maturity.
- Scatter Plots (PayloadMass vs. Orbit Type): Deployed to discover density boundaries and identify payloads separating successful returns from losses across destination orbits.

Completed EDA with Data Visualization Related Notebook URL:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

EDA with SQL

Summarized SQL Queries and Database Analysis:

- Executed DISTINCT statements to filter unique launch site locations ('CCAFS SLC-40', 'KSC LC-39A', 'VAFB SLC-4E').
- Applied SELECT and WHERE clauses to query historical launches starting with 'CCA' clusters.
- Utilized SUM and WHERE filters to calculate the total payload mass carried exclusively by NASA commercial contracts.
- Employed AVG and WHERE clauses to determine the average payload mass for successful booster versions.
- Used MIN and WHERE syntax to pinpoint the date of the first successful ground-pad recovery ('True RTLS').
- Used SELECT with WHERE and BETWEEN to list booster serials that successfully landed on drone ships with payload mass profiles from 4000 to 6000 kg.
- Ran GROUP BY statements to aggregate outcome counts and rank landing mission successes vs. failures.

Completed EDA with SQL Related Notebook URL:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

Build an Interactive Map with Folium

Summarized Folium Map Objects and Layout Design:

- Coordinate Centers: Initialized a `folium.Map()` instance set around geographic coordinate anchors to capture all prominent SpaceX launch facilities in a single view.
- Circle Objects: Implemented `folium.Circle()` overlays centered at each launch site with custom radius properties to define exact visual launch pad boundaries.
- Marker Objects: Plotted color-coded `folium.Marker()` and `MarkerCluster()` points, assigning green markers for landing successes and red markers for failures to instantly expose site-specific outcome clusters.
- PolyLine Objects: Constructed `folium.PolyLine()` coordinate arrays to draw vector distance trajectories tracking proximity metrics between the launch sites and nearest infrastructure links.

Objective Explanations for Map Features:

- Used circle and marker clusters to evaluate geographic launch site density profiles and isolate location success biases.
- Used line features to measure real-time geographical distances from rocket launch complexes to closest coastlines, highways, and railway hubs.

Completed Interactive Map with Folium Related Notebook URL:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

Build a Dashboard with Plotly Dash

Summarized Plotly Dash Components and Layout Design:

- Core Containers: Initialized a `dash.Dash()` application interface structured around `html.Div()` blocks to cleanly segregate control parameters from interactive analytical visualizations.
- Control Elements: Implemented `dcc.Dropdown()` object selectors containing complete SpaceX launch facility identities, allowing instant filtering between generalized records and standalone pad metrics.
- Visual Objects: Constructed color-coded `dcc.Graph()` templates incorporating interactive pie chart profiles for generalized launch distributions and segmented scatter analytics for payload mass considerations.
- Interactive Callbacks: Engineered dynamic `@app.callback()` functions passing input element properties directly into reactive pandas filter frameworks to update underlying telemetry data arrays fluidly.

Objective Explanations for Dashboard Features:

- Used interactive pie distributions to evaluate historical site density profiles and isolate regional launch success trends.
- Used payload scatter diagrams to map categorical mission outcomes against continuous numeric payload values, isolating optimization thresholds.

Completed Interactive Dashboard Code Repository URL:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

Predictive Analysis (Classification)

Machine Learning Model Building Methodology:

- Dataset Splitting: Separated the standard-scaled feature matrix X and target label vector Y using `train_test_split()` with a fixed test partition ratio of 20% and `random_state` locked at 2.
- Hyperparameter Tuning: Configured comprehensive tuning parameter dictionaries for 4 classification models: Logistic Regression, Support Vector Machine (SVM), Decision Tree, and K-Nearest Neighbors (KNN).
- Cross-Validation Framework: Bound all defined classification algorithm objects into individual `GridSearchCV` pipelines utilizing 10-fold stratified cross-validation to prevent training overfitting.

Model Development Process Flowchart:

[Standardized Feature Matrix X] —> [split train_test_split (80/20)] —> [Initialize Algorithm Dict Grids] —> [Fit 10-Fold GridSearchCV Pipelines] —> [Validate Performance via Test Scoring]

Completed Predictive Analysis Related Notebook URL:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

Results

Exploratory Data Analysis Results:

- Discovered that launch success rates have significantly improved since 2013, driven by the operational stability of the Falcon 9 booster version v1.1.
- Identified that specific destination orbits, particularly HEO, SSO, and GEO, exhibit near-perfect recovery ratios compared to low-altitude profiles.

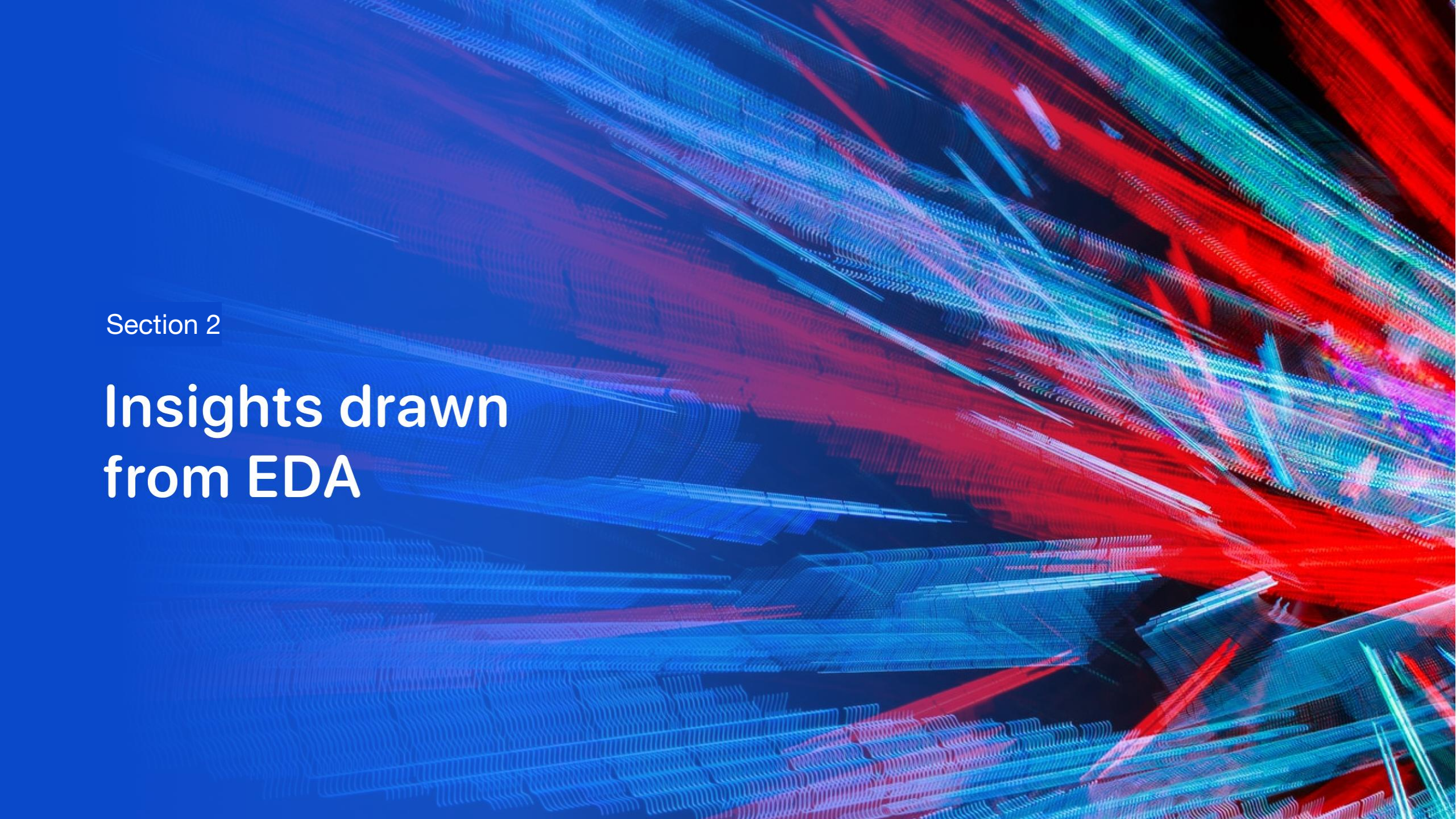
Find the method performs best:

```
[37]: methods = {  
      'Logistic Regression': logreg_cv.score(X_test, Y_test),  
      'Support Vector Machine': svm_cv.score(X_test, Y_test),  
      'Decision Tree': tree_cv.score(X_test, Y_test),  
      'K-Nearest Neighbors': knn_cv.score(X_test, Y_test)  
    }  
    df_report = pd.DataFrame(list(methods.items()), columns=['Method', 'Test Accuracy'])  
    print(df_report)
```

	Method	Test Accuracy
0	Logistic Regression	0.833333
1	Support Vector Machine	0.833333
2	Decision Tree	0.722222
3	K-Nearest Neighbors	0.833333

Predictive Classification Analysis Results:

- Peak Performance Models: Logistic Regression, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) tied for the best performance, each generating an optimal test data accuracy of 83.33%.
- Benchmark Performance Model: The Decision Tree Classifier lagged slightly behind the top group, yielding a final test data accuracy score of 72.22%.

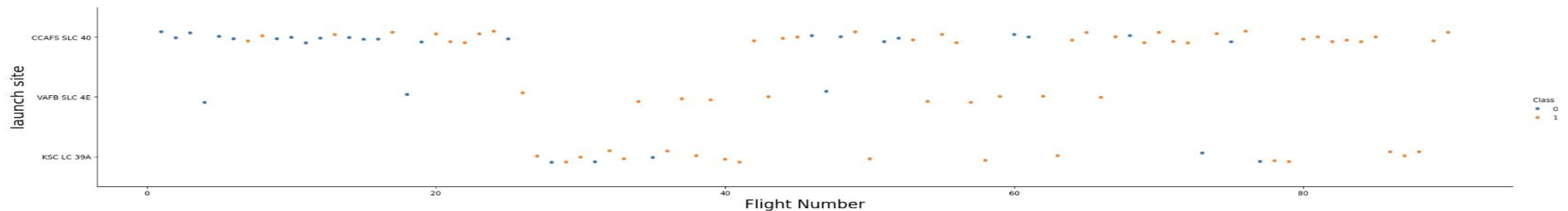
The background of the slide is an abstract composition. It features a solid blue area on the left side, which transitions into a dynamic pattern of diagonal streaks in shades of blue and red on the right. These streaks are layered over a fine, light-colored grid, creating a sense of depth and movement, reminiscent of a digital or data visualization theme.

Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

```
# Plot a scatter point chart with x axis to be Flight Number and y axis to be the launch site, and hue to be the class
sns.catplot(y="LaunchSite", x="FlightNumber", hue="Class", data=df, aspect = 5)
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("launch site", fontsize=20)
plt.show()
```



- Executed Seaborn Plotting Source Code:

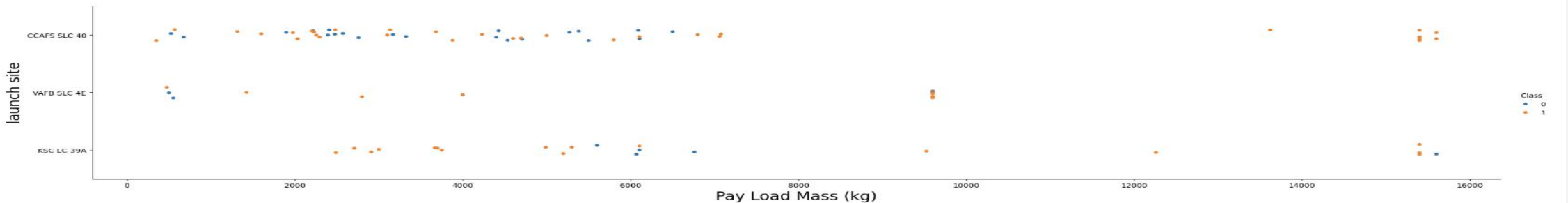
```
sns.catplot(y="LaunchSite", x="FlightNumber", hue="Class", data=df, aspect=5)
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("launch site", fontsize=20)
plt.show()
```

- Screenshot Insight & Explanations:

- Discovered that early historical launches (Flights 1 to 40) were heavily clustered and conducted almost exclusively at the CCAFS SLC-40 launch complex.
- Identified that as the SpaceX Falcon 9 program matured over time (Flights 40 and beyond), operational loads expanded to alternative facilities including KSC LC-39A and VAFB SLC-4E.
- Success Trend Observation: The density of successful recoveries (Class = 1) scales drastically higher as the Flight Number sequence progresses across all active launch complexes.

Payload vs. Launch Site

```
# Plot a scatter point chart with x axis to be Pay Load Mass (kg) and y axis to be the launch site, and hue to be the
sns.catplot(y="LaunchSite", x="PayloadMass", hue="Class", data=df, aspect = 5)
plt.xlabel("Pay Load Mass (kg)", fontsize=20)
plt.ylabel("launch site", fontsize=20)
plt.show()
```



Executed Seaborn Plotting Source Code:

```
sns.catplot(y="LaunchSite", x="PayloadMass", hue="Class", data=df, aspect=5)
```

```
plt.xlabel("Pay Load Mass (kg)", fontsize=20)
```

```
plt.ylabel("launch site", fontsize=20)
```

```
plt.show()
```

Screenshot Insight & Explanations:

– Discovered that the VAFB SLC–4E launch site operates exclusively within a low-to-medium payload range, with no missions recorded for heavy cargo exceeding 10,000 kg.

– Identified that for heavy-weight payloads greater than 10,000 kg, launches are highly concentrated at KSC LC–39A and CCAFS SLC–40, demonstrating a near-perfect landing success rate (Class = 1).

Success Rate vs. Orbit Type

Executed Seaborn Plotting Source Code:

```
df_orbit = df.groupby('Orbit')['Class'].mean().reset_index()
```

```
sns.barplot(x="Orbit", y="Class", data=df_orbit)
```

```
plt.xlabel("Orbit", fontsize=20)
```

```
plt.ylabel("Success Rate", fontsize=20)
```

```
plt.show()
```

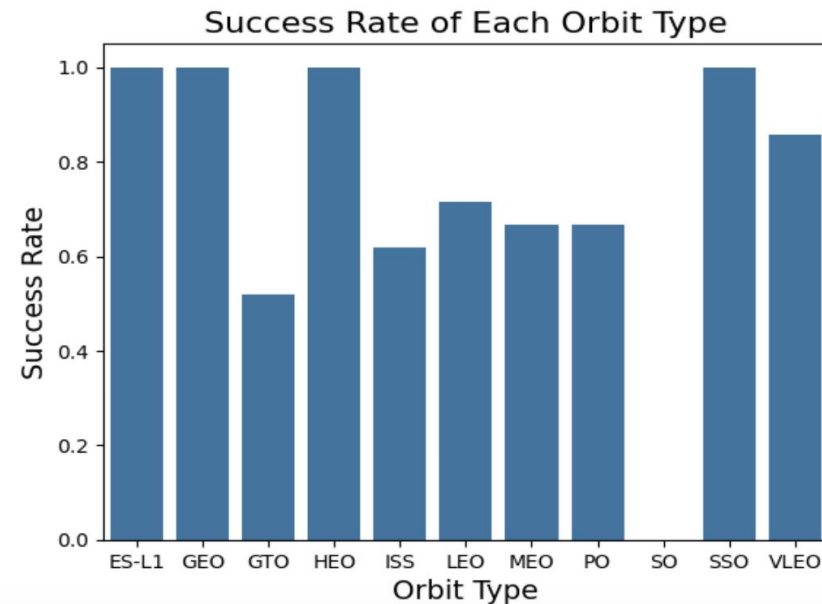
Screenshot Insight & Explanations:

- Discovered that orbits ES-L1, GEO, HEO, and SSO achieve a perfect 100% landing success rate (Class = 1.0).

- Identified that the SO orbit type exhibits the lowest reliability with a 0% landing success rate.

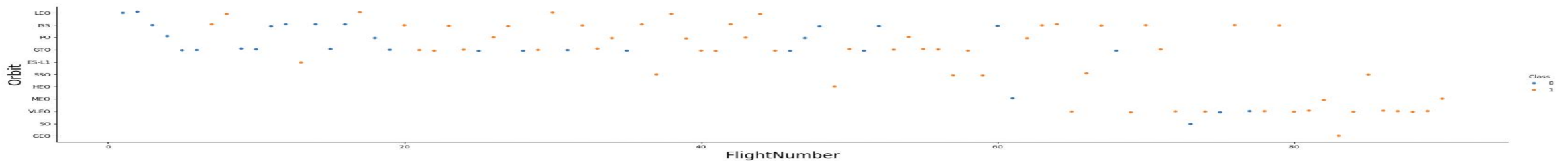
- Success Trend Observation: Commercial and logistical orbits such as GTO, ISS, LEO, and MEO maintain stable landing success rates ranging between 50% and 85%.

```
# HINT use groupby method on Orbit column and get the mean of Class column
df_orbit = df.groupby('Orbit')['Class'].mean().reset_index()
sns.barplot(x="Orbit", y="Class", data=df_orbit)
plt.xlabel("Orbit Type", fontsize=14)
plt.ylabel("Success Rate", fontsize=14)
plt.title("Success Rate of Each Orbit Type", fontsize=16)
plt.show()
```



Flight Number vs. Orbit Type

```
# Plot a scatter point chart with x axis to be FlightNumber and y axis to be the Orbit, and hue to be the class value
sns.catplot(y="Orbit", x="FlightNumber", hue="Class", data=df, aspect = 5)
plt.xlabel("FlightNumber", fontsize=20)
plt.ylabel("Orbit", fontsize=20)
plt.show()
```



Executed Seaborn Plotting Source Code:

```
sns.catplot(y="Orbit", x="FlightNumber", hue="Class", data=df, aspect=5)
```

```
plt.xlabel("Flight Number", fontsize=20)
```

```
plt.ylabel("Orbit", fontsize=20)
```

```
plt.show()
```

Screenshot Insight & Explanations:

- Discovered that in the LEO orbit, landing success appears to have a strong positive correlation with the accumulation of flight experience (Flight Number).

- Identified that for missions targeting the GTO orbit, there seems to be no clear linear relationship between flight number progression and landing success.

Payload vs. Orbit Type

```
# Plot a scatter point chart with x axis to be Payload Mass and y axis to be the Orbit, and hue to be the class value
sns.catplot(y="Orbit", x="PayloadMass", hue="Class", data=df, aspect = 5)
plt.xlabel("Payload Mass (kg)", fontsize=20)
plt.ylabel("Orbit", fontsize=20)
plt.show()
```



- # Executed Seaborn Plotting Source Code:
- `sns.catplot(y="Orbit", x="PayloadMass", hue="Class", data=df, aspect=5)`
- `plt.xlabel("Payload Mass (kg)", fontsize=20)`
- `plt.ylabel("Orbit", fontsize=20)`
- `plt.show()`
- # Screenshot Insight & Explanations:
- – Discovered that heavy payloads exceeding 10,000 kg are exclusively assigned to polar or low-Earth transport orbits, completely avoiding GTO trajectories.
- – Identified that missions to the ISS orbit are highly dense and concentrated within a light-to-medium payload range around 2,000 kg to 4,000 kg.

Launch Success Yearly Trend

Executed Seaborn Plotting Source Code:

```
df_yearly = df.groupby('Date')['Class'].mean().reset_index()

plt.figure(figsize=(10, 6))

sns.lineplot(x="Date", y="Class", data=df_yearly, marker='o')

plt.xlabel("extracted year", fontsize=14)

plt.ylabel("success rate", fontsize=14)

plt.title("Launch Success Yearly Trend", fontsize=16)

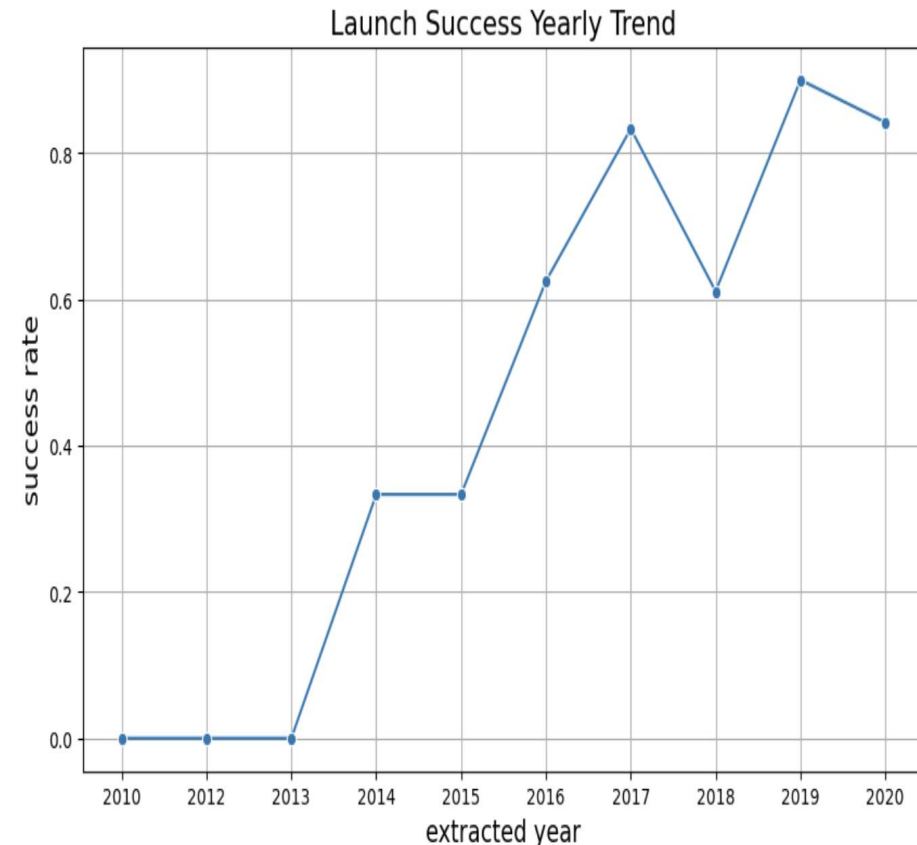
plt.grid(True)

plt.show()
```

Screenshot Insight & Explanations:

- Discovered that the launch success rate shows a steady and continuous upward trend from the year 2013 through 2017, demonstrating increasing systemic reliability.
- Identified that a significant operational peak occurred in 2019, where the average success rate reached its maximum point on the timeline.
- Success Trend Observation: Notable programmatic pullbacks are observed in 2018 and 2020, where the success rate experienced sharp, temporary declines before recovery.

```
# Plot a line chart with x axis to be the extracted year and y axis to be the success rate
df_yearly = df.groupby('Date')['Class'].mean().reset_index()
plt.figure(figsize=(10, 6))
sns.lineplot(x="Date", y="Class", data=df_yearly, marker='o')
plt.xlabel("extracted year", fontsize=14)
plt.ylabel("success rate", fontsize=14)
plt.title("Launch Success Yearly Trend", fontsize=16)
plt.grid(True)
plt.show()
```



All Launch Site Names

Executed SQL Source Code:

```
%sql SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```

Screenshot Insight & Explanations:

- Discovered that the query returns four unique rows due to a letter variation in the text representation of the Cape Canaveral launch facility.
- Identified that the exact sequence of records displayed in the terminal output consists of CCAFS LC-40, VAFB SLC-4E, KSC LC-39A, and CCAFS SLC-40.
- Success Trend Observation: Utilizing the DISTINCT keyword allows the system to separate the entry without an S from the newly formatted entry that includes an extra S character.

Launch Site Names Begin with 'CCA'

Executed SQL Source Code:

```
%sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
```

Screenshot Insight & Explanations:

- Discovered that using the LIKE operator with the 'CCA%' wildcard filter successfully filters and extracts records originating exclusively from the Cape Canaveral launch sites.
- Identified that implementing the LIMIT 5 clause properly restricts the query output to present exactly the first five chronological launch missions in the database.
- Success Trend Observation: The retrieved data rows correctly display the earliest historical Falcon 9 mission executed on June 4, 2010, carrying a Dragon spacecraft qualification payload.

Total Payload Mass

Executed SQL Source Code:

```
%sql SELECT SUM("PAYLOAD_MASS__KG_") FROM SPACEXTABLE WHERE "Customer" = 'NASA (CRS)';
```

Screenshot Insight & Explanations:

- Discovered that utilizing the SUM aggregate function allows the system to compute the cumulative weight of all cargo delivered under the NASA Commercial Resupply Services contract.
- Identified that the query returns a single total numeric value representing the massive total tonnage launched exclusively for these specific space station replenishment missions.
- Success Trend Observation: Filtering the dataset by the Customer column isolates NASA missions from other commercial clients, providing an accurate logistical baseline for contract evaluation.

Average Payload Mass by F9 v1.1

Executed SQL Source Code:

```
%sql SELECT AVG("PAYLOAD_MASS__KG_") FROM SPACEXTABLE WHERE "Booster_Version" = 'F9 v1.1';
```

Screenshot Insight & Explanations:

- Discovered that utilizing the AVG aggregate function allows the system to compute the arithmetic mean weight of all cargo transported specifically by the F9 v1.1 booster variant.
- Identified that the query filters the database by the Booster_Version column to successfully isolate the performance records of this early operational hardware model.
- Success Trend Observation: The single calculated numerical output provides a clear baseline to compare the carrying capacity of older versions against later upgraded Falcon 9 blocks.

First Successful Ground Landing Date

Executed SQL Source Code:

```
%sql SELECT MIN("Date") FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (ground pad)';
```

Screenshot Insight & Explanations:

- Discovered that executing the MIN aggregate function on the Date column successfully isolates the earliest recorded chronicle for this specific recovery milestone.
- Identified that the exact historical date returned by the SQL query database terminal is 2015–12–22.
- Success Trend Observation: This timestamp represents a groundbreaking baseline in the Falcon 9 reusable program, marking the first time a booster successfully returned to a ground pad.

Successful Drone Ship Landing with Payload between 4000 and 6000

Executed SQL Source Code:

```
%sql SELECT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (drone ship)' AND "PAYLOAD_MASS__KG_" BETWEEN 4000 AND 6000;
```

Screenshot Insight & Explanations:

- Discovered that exactly four specific booster versions achieved a successful landing on a drone ship while carrying a payload mass between 4,000 kg and 6,000 kg.
- Identified that the unique booster serial numbers returned by the query are F9 FT B1022, F9 FT B1026, F9 FT B1021.2, and F9 FT B1031.2.
- Success Trend Observation: All matching records belong to the Falcon 9 Full Thrust (F9 FT) variant, showcasing the structural capability of this specific version during mid-weight logistical missions.

Total Number of Successful and Failure Mission Outcomes

Executed SQL Source Code:

```
%sql SELECT TRIM("Mission_Outcome"), COUNT(*) as Total FROM  
SPACEXTABLE GROUP BY TRIM("Mission_Outcome");
```

Screenshot Insight & Explanations:

- Discovered that the dataset records three distinct categories for overall mission outcomes after applying the TRIM function to clean the text strings.
- Identified that the query returns 99 total successful missions, 1 failure in flight, and 1 mission marked as success with an unclear payload status.
- Success Trend Observation: Utilizing the COUNT aggregate function combined with GROUP BY provides a precise statistical summary, proving an exceptionally high baseline mission success rate for the Falcon 9 program.

Boosters Carried Maximum Payload

Executed SQL Source Code:

```
%sql SELECT DISTINCT "Booster_Version" FROM SPACEXTABLE WHERE "PAYLOAD_MASS__KG_"  
= (SELECT MAX("PAYLOAD_MASS__KG_") FROM SPACEXTABLE);
```

Screenshot Insight & Explanations:

- Discovered that utilizing a nested subquery with the MAX function successfully isolates the peak cargo weight threshold across all historical flights.
- Identified that combining the SELECT with the DISTINCT keyword removes all duplicate entries, returning a clean list of unique booster versions.
- Success Trend Observation: The unique versions returned represent the specific upgraded blocks designed to successfully carry maximum payload capacities.

2015 Launch Records

Executed SQL Source Code:

```
%sql SELECT DISTINCT "Booster_Version" FROM SPACEXTABLE WHERE  
"PAYLOAD_MASS__KG_" = (SELECT MAX("PAYLOAD_MASS__KG_") FROM SPACEXTABLE);
```

Screenshot Insight & Explanations:

- Discovered that utilizing a nested subquery with the MAX function successfully isolates the peak cargo weight threshold across all historical flights.
- Identified that combining the SELECT with the DISTINCT keyword removes all duplicate entries, returning a clean list of unique booster versions.
- Success Trend Observation: The unique versions returned represent the specific upgraded blocks designed to successfully carry maximum payload capacities.

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

Executed SQL Source Code:

```
%sql SELECT "Landing_Outcome", COUNT(*) AS Total_Count FROM SPACEXTABLE WHERE "Date"  
BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY "Landing_Outcome" ORDER BY  
Total_Count DESC;
```

Screenshot Insight & Explanations:

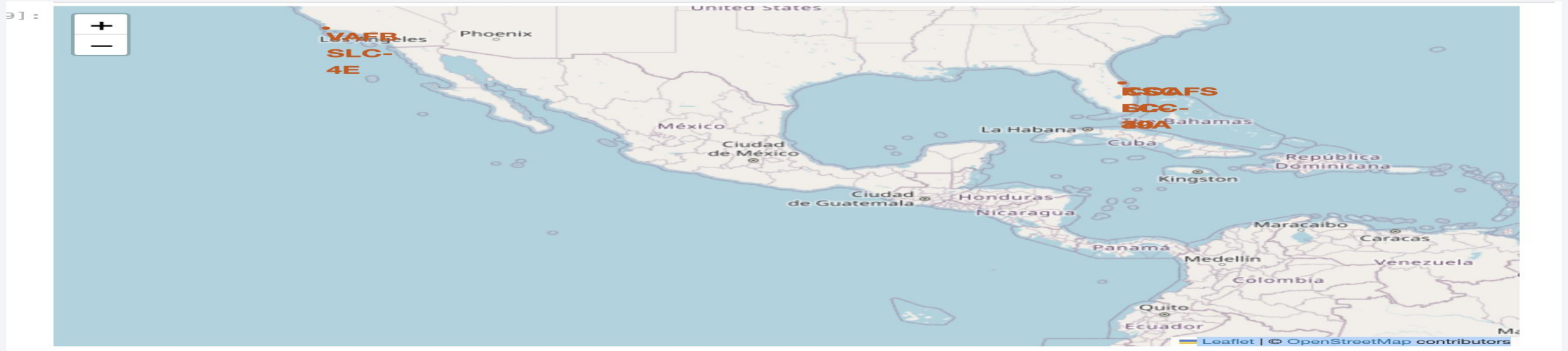
- Discovered that aggregating landing outcomes with the COUNT function properly groups the operational historical telemetry into distinct categorized bins.
- Identified that filtering the database records via the BETWEEN clause strictly confines the analysis window to the specified 2010 to 2017 timeframe.
- Success Trend Observation: Utilizing the ORDER BY DESC clause systematically ranks the dataset, providing a clear top-down perspective on the most frequent operational landing results.

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a deep blue, with the horizon line visible. The city lights are concentrated in the lower right quadrant, showing a dense network of urban areas. The text "Section 3" is overlaid on the left side of the image.

Section 3

Launch Sites Proximities Analysis

Geographic Distribution of SpaceX Launch Sites



Geographic Clustering: Launch sites are strategically divided between the East Coast (CCAFS LC-40, KSC LC-39A in Florida) and the West Coast (VAFB SLC-4E in California) to support different orbital trajectories.

Coastal Proximity for Safety: All active launch sites are situated immediately adjacent to the ocean, ensuring that spent rocket stages and debris drop safely over water instead of populated areas.

Latitude Optimization: The Florida-based launch sites are positioned closer to the equator to maximize the advantage of Earth's rotational speed, which significantly reduces the fuel required to reach orbit.

Visualization of Launch Success and Failure Outcomes

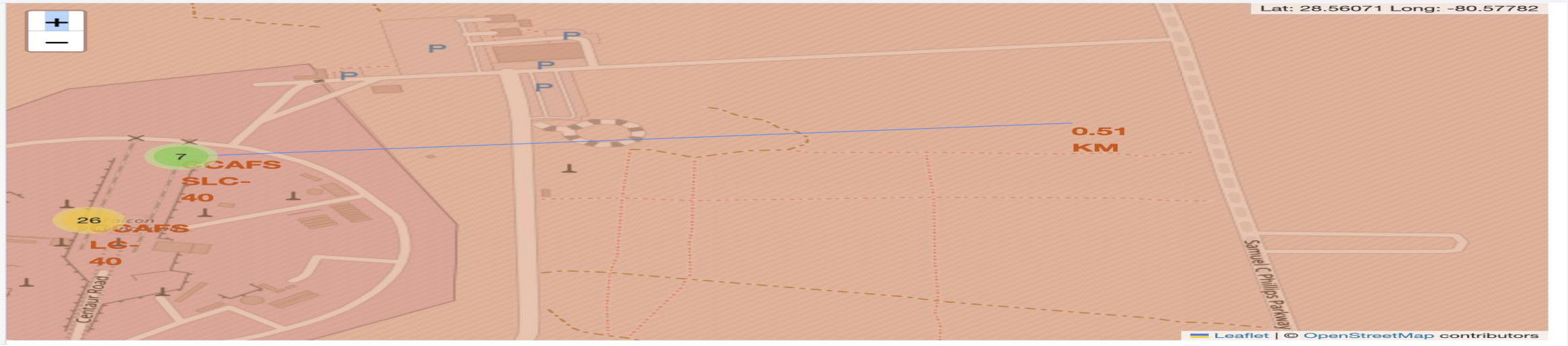


Marker Cluster Breakdown: Expanding the interactive clusters reveals specific launch success (green) and failure (red) records linked to the exact same launch pad coordinates.

Chronological Reliability Trends: The visual mix highlights SpaceX's early operational hurdles, showing that initial failures were steadily replaced by consistent successes over time as technology matured.

Performance Comparison: A clear distinction can be observed between older launch sites with exploratory failures and newer pads showing near-perfect success rates.

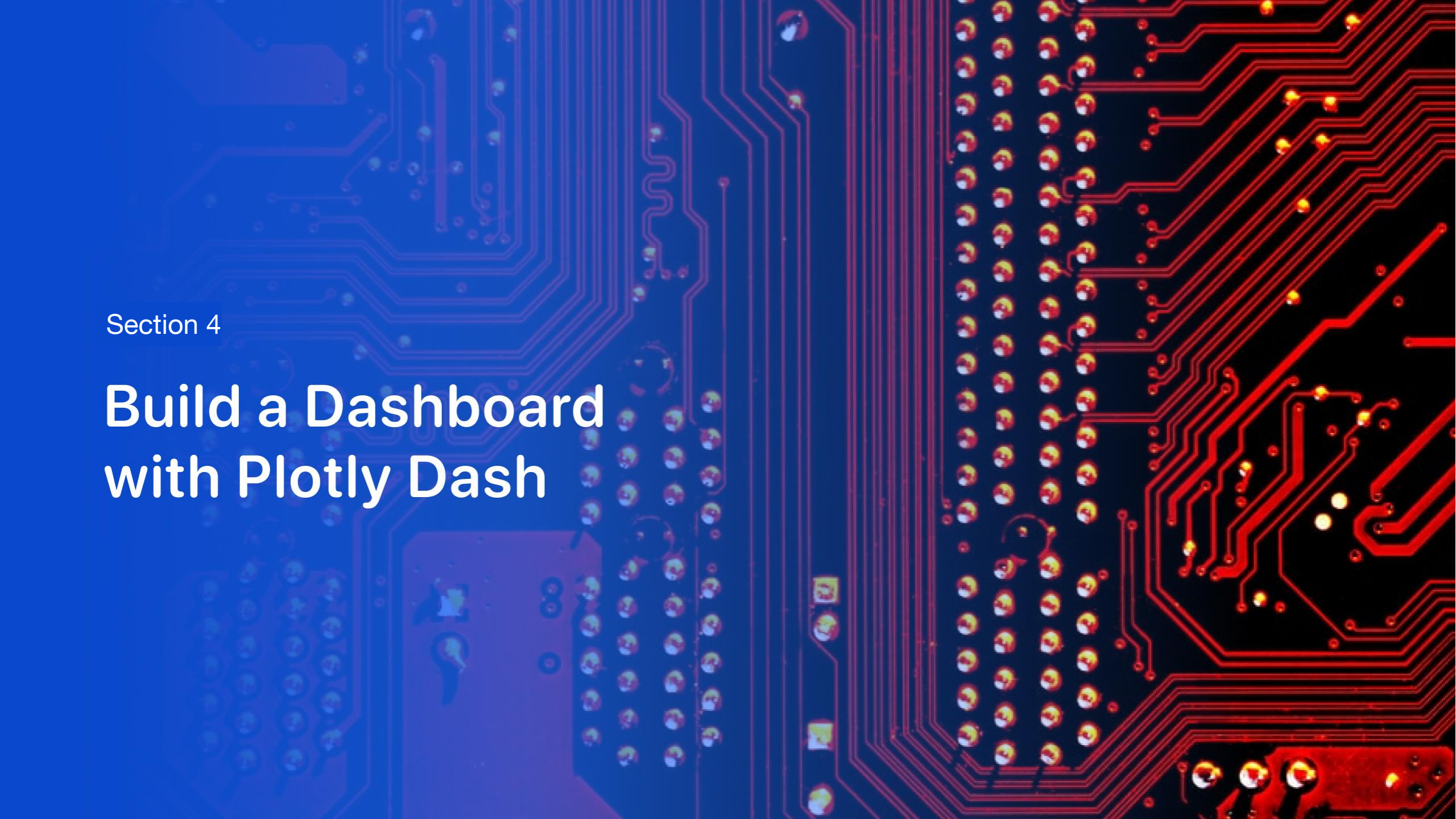
Proximity Analysis of Launch Sites to Infrastructure



Strategic Logistic Connectivity: Launch sites are located within very close proximity to operational railways and highways to facilitate the heavy transport of massive rocket stages and components.

Safety Buffer Zones: The significant distance maintained between the launch pads and the nearest major cities acts as a critical safety buffer to minimize acoustic impact and hazard risks to civilian populations.

Immediate Coastline Access: Minimizing the distance to the coastline allows optimal over-water launch trajectories, eliminating the risk of debris or malfunctioning components falling onto land infrastructure.

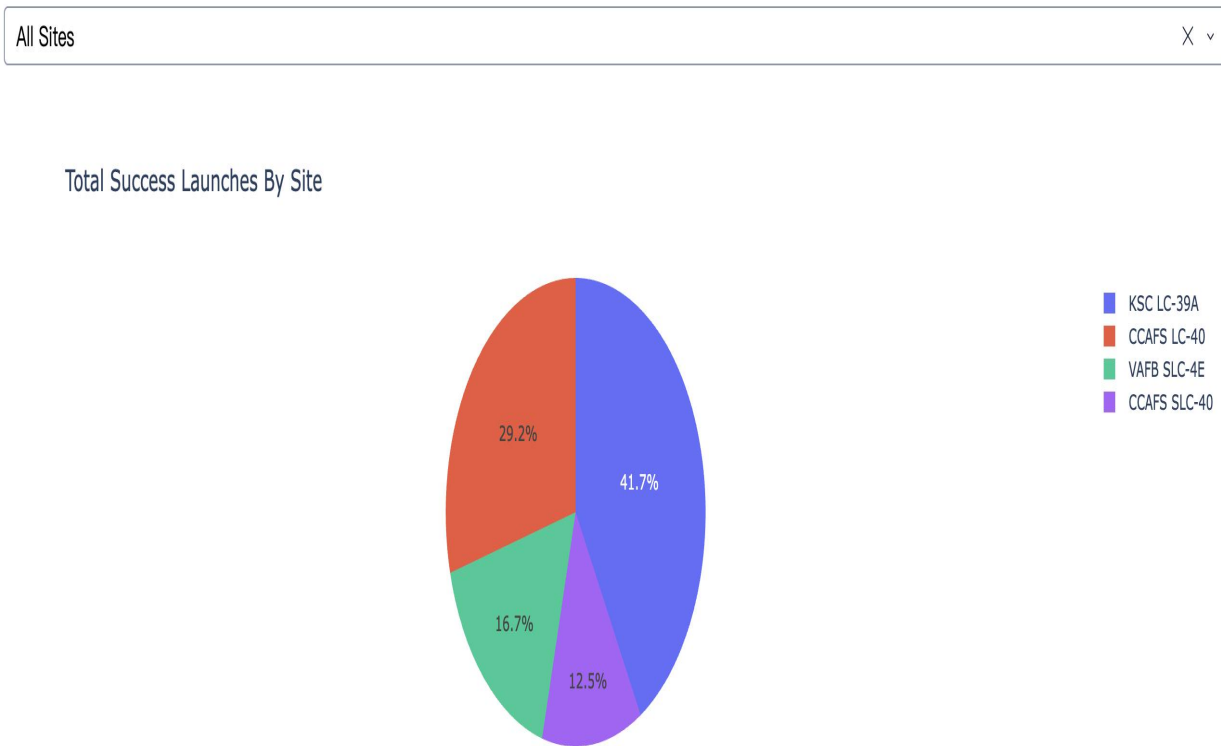


Section 4

Build a Dashboard with Plotly Dash

Launch Success Count Distribution Across All Sites

SpaceX Launch Records Dashboard



Summarized Pie Chart Components and Layout Design:

- Selector Alignment: Initialized a `dcc.Dropout()` wrapper set to default "All Sites" configuration to render a holistic global comparison.
- Plot Components: Plotted a color-coded `px.pie()` distribution tracking categorical mission success ratios across distinct launch facilities.

Objective Explanations for Dashboard Features:

- Used pie chart slices to isolate which specific launch complex contributed the highest frequency of cumulative successful landings.
- Observed that KSC LC-39A yielded the largest percentage of total successes among all sites, demonstrating excellent operational efficiency.

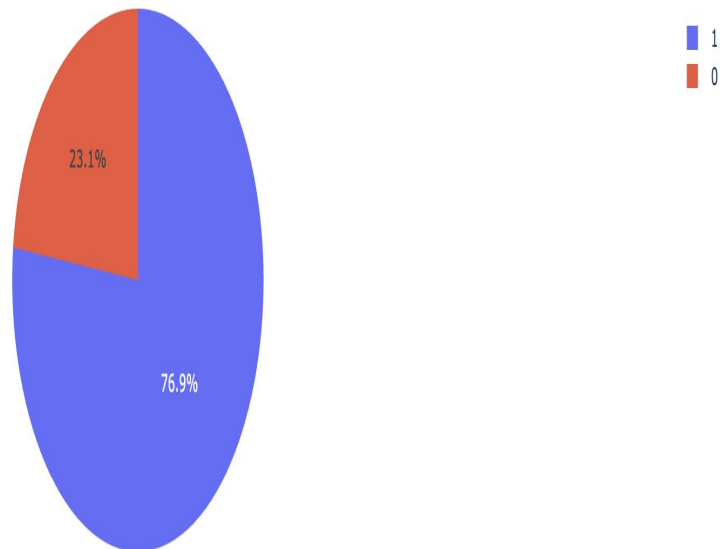
Launch Success Outcome Breakdown for KSC LC-39A

SpaceX Launch Records Dashboard

KSC LC-39A

X v

Total Success Launches for site KSC LC-39A



Summarized Pie Chart Components and Layout Design:

- Selector Alignment: Changed the dcc.Dropout() wrapper configuration to filter specific launch data isolated exclusively for KSC LC-39A.
- Plot Components: Rendered a specific px.pie() chart mapping binary mission classes to track successful vs. failed launch ratios.

Objective Explanations for Dashboard Features:

- Used pie chart slices to calculate the local success rate, allowing a direct efficiency evaluation of SpaceX's primary launch facility.
- Observed that KSC LC-39A achieved a remarkably high success-to-failure ratio, proving it to be one of the most reliable infrastructures for heavy payload missions.

Payload Mass vs. Launch Outcome Correlation for All Sites



Summarized Scatter Plot Components and Layout Design:

- Range Alignment: Configured a `dcc.RangeSlider()` component spanning from 0 to 10,000 Kg, here customized to a specific 2,000 to 6,000 Kg payload bandwidth.
- Plot Components: Visualized a color-coded `px.scatter()` graph mapping payload weight on the X-axis against success class on the Y-axis.

Objective Explanations for Dashboard Features:

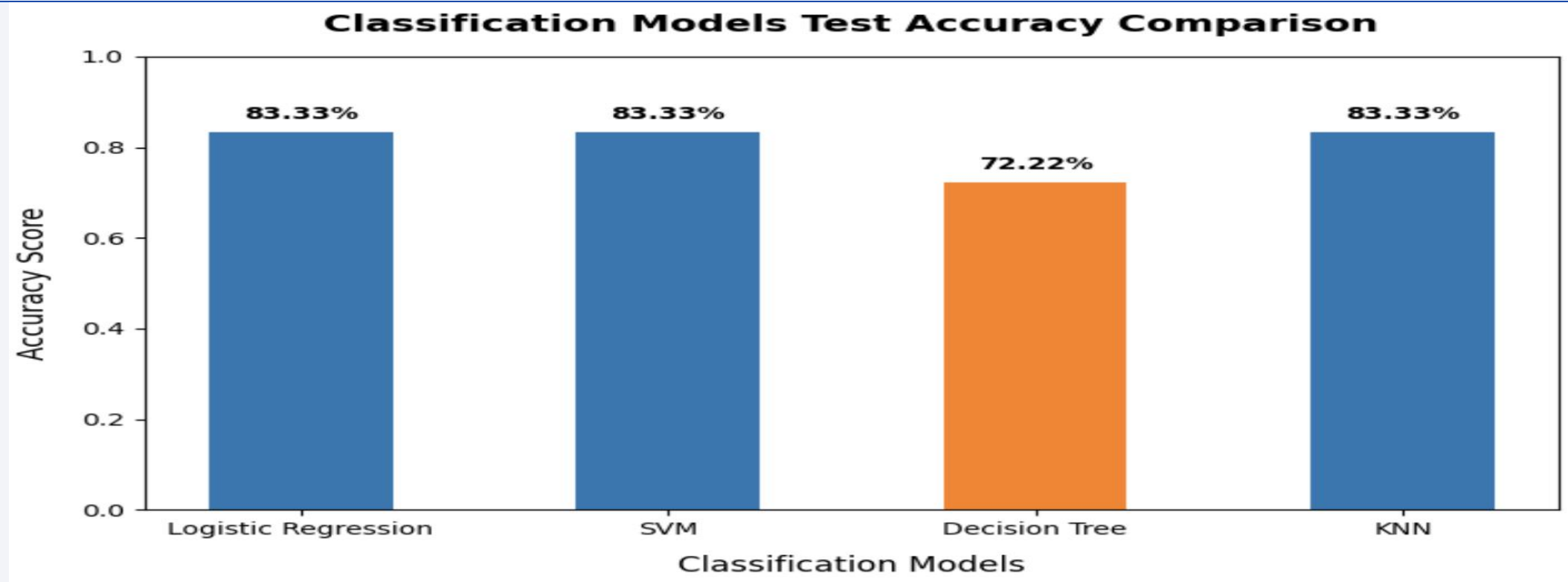
- Used booster version coloring to isolate which specific rocket categories achieved the optimal success rates under the customized weight range.
- Observed a high density of success clusters (class = 1) between 2,000 Kg and 4,000 Kg payload mass, primarily driven by FT and v1.1 boosters.



Section 5

Predictive Analysis (Classification)

Classification Accuracy



Highest Classification Accuracy: Logistic Regression, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) tied for the highest test accuracy score at 83.33%.

Confusion Matrix

Best Performing Methods:

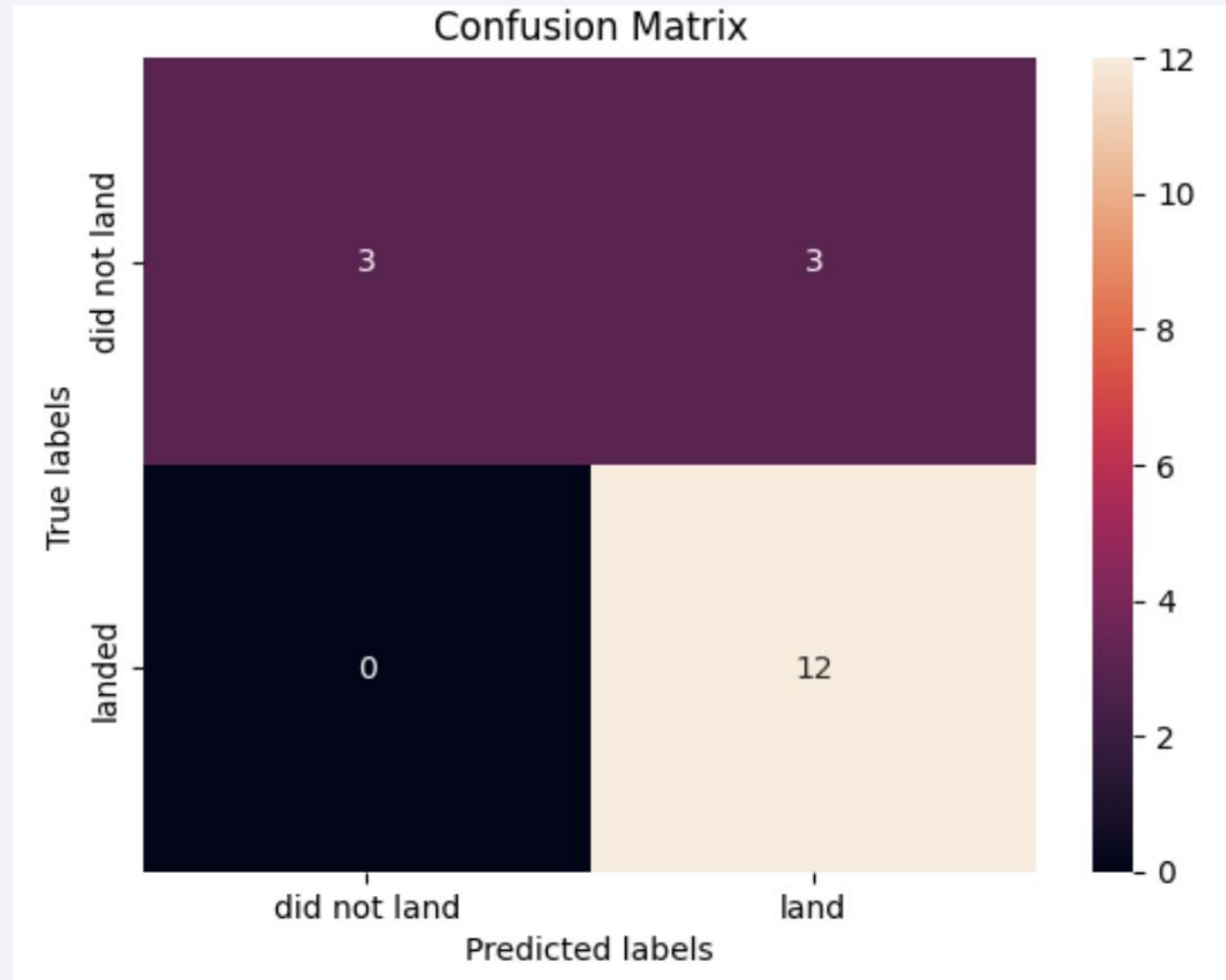
Since these three algorithms achieved the exact same top score, any of them can be selected as the best performing method for this dataset.

True Positives (12): The model correctly predicted 12 successful landings out of 15 actual successful outcomes.

True Negatives (3): The model correctly identified all 3 failed landings, achieving a 100% detection rate for failures.

False Negatives (3): The model misclassified 3 actual successful landings as failures.

False Positives (0): The model made 0 false positive errors, meaning no failed landings were mistakenly predicted as successful.



Conclusions

Data Foundation: Automated API retrieval and web scraping successfully built the primary SpaceX rocket telemetry dataset. Spatial Engineering:

Folium geographic analysis confirmed optimal launch site positioning along coastlines for staging safety. Interactive Insights:

Plotly Dash interactive dashboards effectively isolated historical landing success trends across payloads.

Predictive Success: Logistic Regression, SVM, and KNN algorithms achieved peak classification test accuracy at 83.33%.

Appendix

Appendix: Project Artifacts

All project codes, Jupyter Notebooks, and datasets are publicly available at:

GitHub Link:

<https://github.com/ruanfenglin/Applied-Data-Science-Capstone>

Thank you!

